

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based on your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1

input_string = "your_first_name_here"
result = hash_to_range(input_string)
print(result)

2

hash_to_range("Lian")

1
```

Starter Code for Question 1

```
# Definition for a binary tree node.
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right
#def is_duplicate(root: TreeNode) -> int:
# TODO
def is_duplicate(root):
    if not root:
        return -1

    seen = {}
    duplicates = {}

    def dfs(node, distance):
        if not node:
            return

        if node.val in seen:
            # Found duplicate
            min_dist = min(seen[node.val], distance)
            if node.val in duplicates:
                duplicates[node.val] = min(duplicates[node.val],
min_dist)
            else:
                duplicates[node.val] = min_dist
        else:
            seen[node.val] = distance

        dfs(node.left, distance + 1)
        dfs(node.right, distance + 1)

    dfs(root, 0)

    if not duplicates:
        return -1

    # Return duplicate with minimum distance
    return min(duplicates.items(), key=lambda x: x[1])[0]

root1 = TreeNode(1)
root1.left = TreeNode(2)
root1.right = TreeNode(2)
root1.left.left = TreeNode(3)
root1.left.right = TreeNode(5)
root1.right.left = TreeNode(6)
root1.right.right = TreeNode(7)
```

```

print(is_duplicate(root1))

2

root2 = TreeNode(1)
root2.left = TreeNode(10)
root2.right = TreeNode(2)
root2.left.left = TreeNode(3)
root2.left.right = TreeNode(10)
root2.right.left = TreeNode(12)
root2.right.right = TreeNode(12)

print(is_duplicate(root2))

10

root3 = TreeNode(10)
root3.left = TreeNode(9)
root3.right = TreeNode(7)
root3.left.left = TreeNode(8)
print(is_duplicate(root3))

-1

```

Starter Code for Question 2

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
    # TODO

```

Starter Code for Question 3

```

def missing_num(nums: List) -> int:
    # TODO

```

Part 2:

You and your partner must share each other's Assignment 1 submission.

My partner is : "kyuhwan" , Kyuhwan Kim

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
# My partner is Assignment 1 is to find the first number that is duplicated in minimum distance in a list of integers.
```

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

My new example:

Input: `nums = [1, 20, 5, 5, 4, 9]`

Output: `5`

My partner's Examples

Input: `nums = [10, 20, 30, 40, 10, 50]`

Output: `10`

Input: `nums1 = [1, 2, 3, 4, 5, 6]`

Output: `-1`

My partner created two lists. First list is as follows: 10,20, 30, 40, 10, 50. First he create a empty set as seen. Then for each integer in the list, he would compare if the integer is in the set:seen or not. If it is, it would return the integar. If not, it would add it into the set :seen. if it went throught the list and can't find any duplicate, it will return -1.

so first it takes 10, and it's not in the set: seen, so it adds 10 to the set seen. Then it takes 20, and it's not in the set:seen, so it adds 20 to the set seen. [10, 20]. Then it takes the third element 30, and it's not in the set: seen. so it adds 30 to the set seen. [10,20,30], then it takes 40, and it's not in the set:seen , so it adds 40 to the set seen. [10, 20, 30,40]. Then it takes the fifth element 10, and it is in the set:seen, so it returns 10 as the answer.

```
# Your answer here
```

- Copy the solution your partner wrote.

```
# Your answer here
```

```
nums = [10, 20, 30, 40, 10, 50] #answer is 10
```

```
nums1 = [1, 2, 3, 4, 5, 6] #answer is -1
```

```
def first_duplicate(nums: List[int]) -> int:
    seen = set()
    for num in nums:
```

```

        if num in seen:
            return num
        seen.add(num)
    return -1

print(first_duplicate(nums))
print(first_duplicate(nums1))

```

- Explain why their solution works in your own words.

Your answer here

The solution create a queue to perform a level-order traversal. It starts by initializing the queue with the first integer in the list. Then it enters a loop that continues until it finds the first duplicate or the queue is empty. In each iteration, it dequeues an integer from the front of the queue, and check if it has been seen before by checking if it is in the set:seen. if it has, it returns the integer as the first duplicate. If not, it added it into the set:seen and continues the process.

- Explain the problem's time and space complexity in your own words.

Your answer here

The problem's time is $O(n)$ and space is $O(n)$

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

Your answer here

My partner's solution is efficient as it only requires a single pass through the list of integers, resulting in a time complexity of $O(n)$. Adding elements to a set and checking for membership in a set both have an average time complexity of $O(1)$, making the overall approach optimal for the problem.

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

Your answer here

The assignments are interesting both from creator and reviewer perspective. Both assignments requires to understand the sorting and searching logic and apply the search algorithm to find any match. Both can return as soon as the condition is met (duplicate found or

invalid bracket). My assignment 1 the match target is pre-defined while my partner's assignment 1, the search target is dynamic which would require to store the intermedial as a moving search list. Both assignment algorithms involve creating a queue to track the progress and involve recursive loop statement.

Both problems showcase different aspects of algorithmic thinking. The bracket problem emphasizes order preservation and pattern matching. The duplicate problem emphasizes efficient membership testing and early termination. The duplicate problem was about finding existence efficiently, while the bracket problem was about maintaining and validating structural relationships. Both assignments demonstrate how fundamental data structures like stacks perfectly capture real-world patterns - in this case, the "last opened, first closed" nature of bracket sequences and "first match first closed" duplicate search. Both solutions demonstrate how clean, readable codes that can directly maps to the problem statements, making them easy to understand and maintain.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

▮ **Please review our [Assignment Submission Guide](#)** ▮ for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at [#cohort-6-help](#). Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

